OPERATING SYSTEMS CASE STUDY - FINAL IMPLEMENTATION REPORT

Predictive Deadlock Prevention Using Contention Analysis and Score Forecasting

Measured experiment, integrated website, and evaluation

Team: Midhun Gereketi (frontend and integration); P. Satya Mani Shraavanth (threaded backend and API); M. Lakshman Sai (analysis and evaluation).

Abstract

A controlled experiment shows how opposite acquisition orders create a two-thread deadlock and how an observed contention signal can activate a shared lock order. Real Python threads and mutexes produce timestamped events. A Wait-For Graph confirms cycles; a heuristic score warns before the cycle in a staged schedule. A FastAPI website renders the trace, score, heatmap, and repeated-trial comparison. In the saved sample, all 10 vulnerable runs deadlocked and all 10 protected runs completed. The result describes this controlled two-lock experiment, not arbitrary applications.

1. Problem and objectives

T1 requests locks A then B; T2 requests B then A. If each holds its first lock and requests the other, neither can proceed. The project records this dependency, distinguishes risk from confirmed deadlock, and applies a learned common order before protected workers acquire their main-work locks.

2. Implemented system

Module	Implementation and output
Threaded experiment	Vulnerable and protected Python-thread trials with bounded mutex waits
Observer	Timestamped request, acquisition, wait, release, warning, and terminal events
Analysis	Wait-For Graph, cycle detection, score trend, contention matrix
API and website	FastAPI trial/batch endpoints and interactive browser dashboard
Evaluation	Per-trial results, aggregate rates, warning lead, latency, wait and runtime

The repository includes source code, tests, raw results, importable example trials, and a presentation guide. The integrated app runs locally or at https://predictive-deadlock-lab.vercel.app/.

3. Detection, warning, and prevention

Wait-For Graph

An edge T1 to T2 exists only when T1 waits for a lock currently owned by T2. With one instance of each lock, T1 to T2 and T2 to T1 form a cycle that confirms deadlock. A score warning alone does not confirm a cycle.

Threat score

The base 0-100 score combines normalized blocked-thread count (35%), observed wait duration (30%), contention count (20%), and resource conflict state (15%). The default high-risk threshold is 70. In this two-lock case, when A and B are held by distinct threads and one cross-resource wait exists, a structural rule raises the score to at least 72. It uses only state available at that timestamp. A confirmed cycle is marked separately.

Adaptive order

The protected run first creates a bounded, cycle-free probe: T1 holds A, T2 holds B, and T1 waits for B. The observed held/requested pair derives the total order A then B. T2 releases B before requesting A, so the probe never forms a cycle. The main protected workers then normalize both requested lock lists to the derived order before acquiring any main-work lock. An alternate observed B-to-A dependency would derive B then A.

Why this prevents the demonstrated cycle

When every worker acquires A before B, no worker can hold B while waiting for A. Thus the opposite edge needed for a circular wait cannot form in the protected main phase.

4. Experimental method

The saved comparison contains 10 vulnerable and 10 protected trials on Windows with Python. Each trial has two logical worker roles and single-instance mutexes A and B. A barrier controls the vulnerable interleaving. A staged 120 ms interval lets the observer record a warning after T1 begins waiting and before T2 requests A. The protected run includes a cycle-free probe in its measured runtime. Trial waits are bounded, and comparison calls use a 3-second per-trial ceiling. The exact collection timestamp and runtime environment are in docs/results.json.

Metrics

Deadlock rate counts trials with an observed WFG cycle. Completion rate counts trials in which both workers finish. Warning lead is the time from the first pre-cycle high score to first cycle observation. Detection latency is the time from first cycle observation to the supervisor notification. Inconclusive runs are counted separately. Full trial runtime includes cleanup after deadlock notification; the dashboard trial timeline ends at the notification.

5. Measured results

Saved 10-trial-per-mode dataset

Metric	Vulnerable	Protected
Trials	10	10
Confirmed deadlocks	10	0
Completed trials	0	10
Inconclusive trials	0	0
Deadlock rate	100.0%	0.0%
Mean full runtime	813.6 ms	104.6 ms
Mean peak wait	809 ms	28.9 ms
Mean peak score	96	72
Mean warning lead	120.4 ms	Not applicable
Mean detection latency	4.6 ms	Not applicable

The protected score peaks at 72 during its safe probe: this is a warning followed by prevention, not a protected deadlock. The vulnerable run continues through bounded waits and cleanup after the cycle is detected. That cleanup explains why full runtime is longer than the notification timeline.

6. Interpretation and limitations

The schedule deliberately makes the vulnerable cycle reproducible and provides an observation interval. The warning lead is therefore evidence about this staged case, not a measured general prediction rate. The learned order applies to the protected main phase; it does not repair locks already held by a running worker. The study has only two threads, two locks, one machine, and no larger-workload throughput test. Trials share the FastAPI process rather than isolated worker processes, although mutex waits and the supervisor are bounded.

7. Reproduce and present

Step	Command or action
Install	pip install -r member2_backend/requirements.txt
Python tests	python -m pytest member2_backend/tests member3_analysis/tests -q
Frontend tests	node --test frontend/tests/trial-data.test.js
Start local app	python -m uvicorn member2_backend.api:app --host 127.0.0.1 --port 8000
Open dashboard	http://127.0.0.1:8000/
Regenerate data	python scripts/collect_results.py --trials 10

Run a live vulnerable trial, then a live protected trial. On the Compare page, run the five-each batch and read the measured table. The two docs/sample-*.json files are fallback examples that can be imported if network access is unavailable. The integrated hosted FastAPI app is https://predictive-deadlock-lab.vercel.app/.

8. Files and evidence

The repository contains member2_backend/ for threaded experiments and API, member3_analysis/ for WFG/scoring/comparison, frontend/ for visualization, docs/results.json for aggregate and per-trial measurements, and docs/sample-vulnerable.json and docs/sample-protected.json for reproducible dashboard imports. The exact study protocol and additional limitations are documented in docs/FINAL_REPORT.md.

9. Conclusion

The prototype reproduces and detects a controlled circular wait, records a warning before the cycle in the staged vulnerable run, and completes both workers when a shared lock order is derived and applied before their protected main work. Repeated measurements support that conclusion for this scenario. Extending it to larger workloads would require broader schedules, more resources, false-warning analysis, and process isolation.